PROJET AWD · VALIDATION INTERMÉDIAIRE

MediCare

Plateforme distribuée de gestion de clinique, en architecture microservices

Emna Laouini · Dhouha Belhaj · Eslem Zen
Sarra Akremi · Hiba Dogui · Maram Ghazel

Classe 4DAS-I14 · Groupe [gX]
ESPRIT · 2026–2027

CONTEXTE ET PROBLÉMATIQUE

Le parcours patient est encore éclaté

Rendez-vous par téléphone

Créneaux mal gérés, doubles réservations, attente.

Dossiers dispersés

Papier et fichiers locaux, peu de partage entre médecins.

Facturation manuelle

Erreurs, retards et aucun suivi pour le patient.

NOTRE RÉPONSE

Une plateforme unique, évolutive et sécurisée, où chaque domaine métier est un microservice indépendant.

ACTEURS ET CAS D'UTILISATION

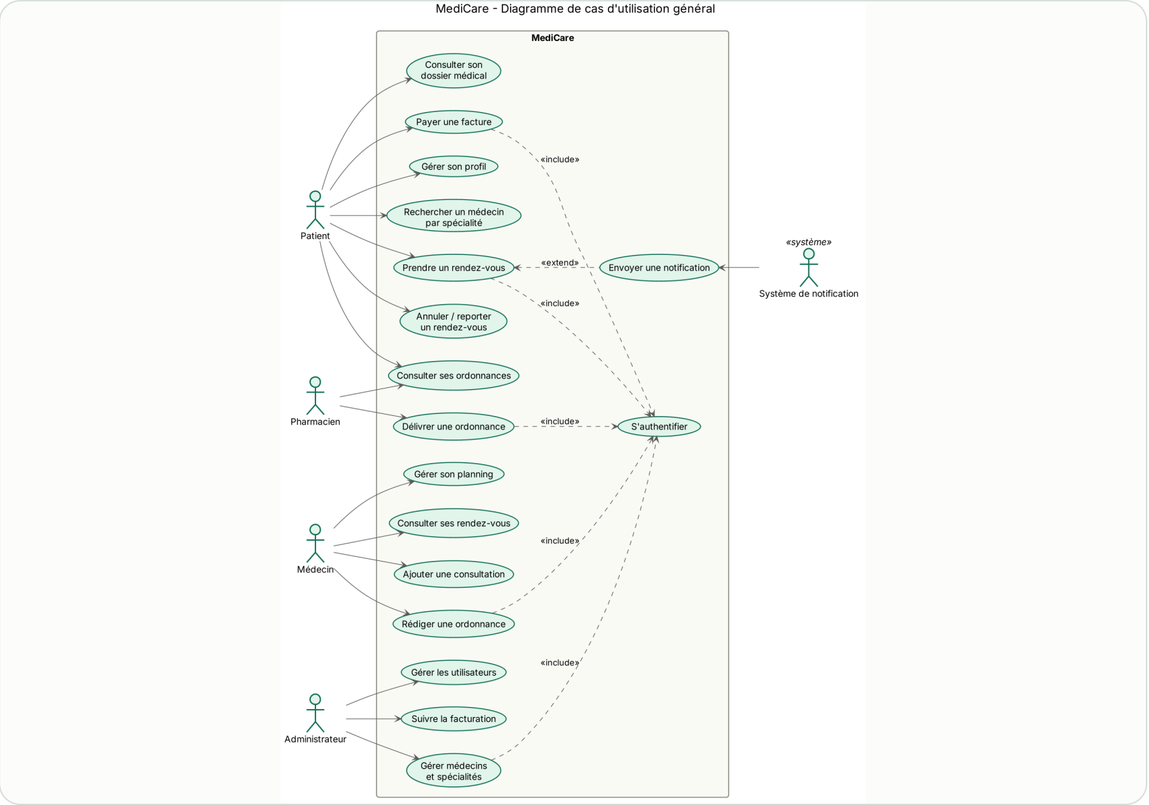
Quatre acteurs, un parcours commun

Patient
Prend rendez-vous, consulte son dossier, ses ordonnances et paie ses factures.

Médecin
Gère son planning, ajoute des consultations, rédige les ordonnances.

Pharmacien
Consulte et délivre les ordonnances.

Administrateur
Gère médecins, spécialités, utilisateurs et facturation.



FONCTIONNALITÉS PRINCIPALES

Ce que MediCare permet de faire

Patients
Profil, groupe sanguin, allergies

Médecins
Spécialités, créneaux, recherche

Rendez-vous
Réserver, annuler, reporter

Dossier médical
Consultations, antécédents, documents

Ordonnances
Médicaments, posologie, délivrance

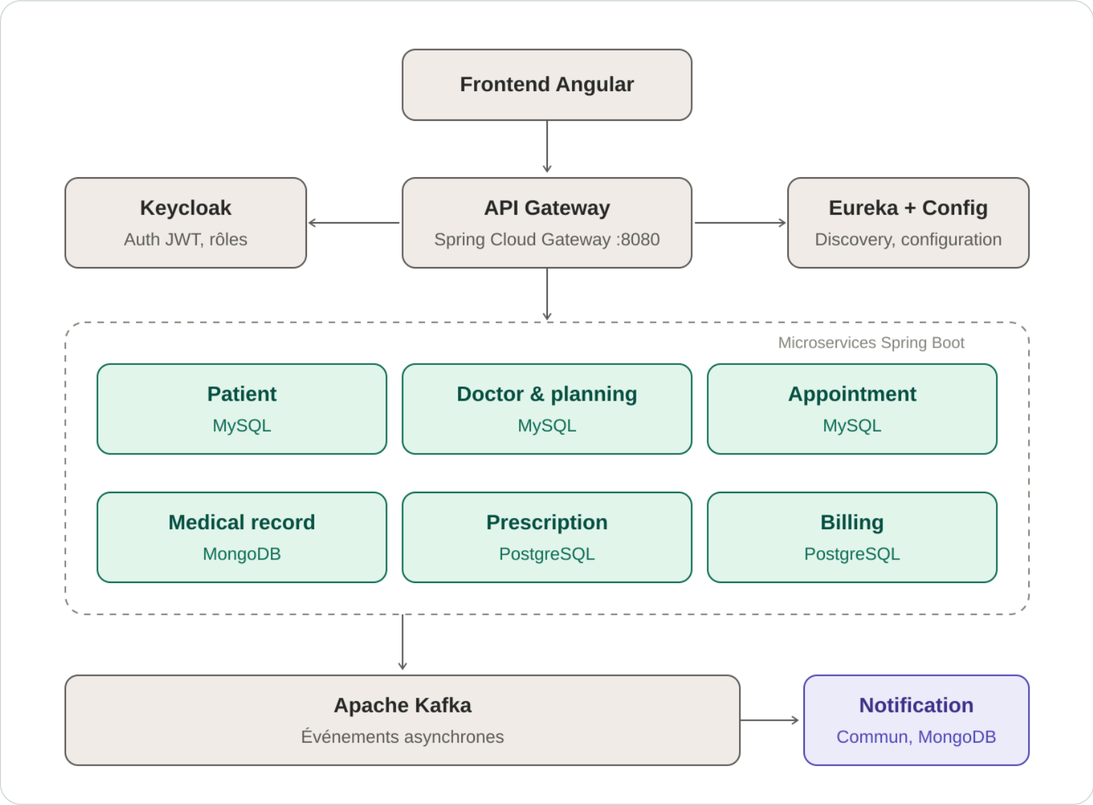
Facturation
Facture auto, paiement, reçu

Notifications
Confirmation, rappel, annulation

Sécurité
SSO Keycloak, rôles, JWT

ARCHITECTURE GLOBALE

Spring Cloud, de la Gateway à Kafka



Point d’entrée unique

Angular ne parle qu’à l’API Gateway.

Sécurité centralisée

Keycloak émet le JWT, la Gateway le valide.

Découverte dynamique

Eureka localise chaque service, Config Server centralise la configuration.

Database per service

Chaque microservice possède sa base.

Événements

Kafka découple les traitements asynchrones.

SERVICES D’INFRASTRUCTURE

Les briques communes à toute l’équipe

Composant	Technologie	Rôle	Port
API Gateway	Spring Cloud Gateway	Routage, validation JWT, CORS	8080
Discovery	Eureka Server	Enregistrement et découverte des services	8761
Configuration	Spring Cloud Config	Configuration centralisée	8888
Sécurité	Keycloak	OAuth2 / OpenID Connect, rôles	8180
Messaging	Apache Kafka	Bus d’événements asynchrones	9092
Déploiement	Docker Compose	Lancement de toute l’architecture	–

DÉCOUPAGE EN MICROSERVICES

Six services métier et un service commun

Microservice	Responsabilité	Base	Responsable
patient-service	Profils des patients	MySQL	Emna Laouini
doctor-service	Médecins, spécialités, créneaux	MySQL	Dhouha Belhaj
appointment-service	Réservation et annulation des rendez-vous	MySQL	Eslem Zen
medical-record-service	Consultations, antécédents, documents	MongoDB	Sarra Akremi
prescription-service	Ordonnances et délivrance	PostgreSQL	Hiba Dogui
billing-service	Factures et paiements	PostgreSQL	Maram Ghazel
notification-service	Notifications par événements (commun)	MongoDB	Toute l’équipe

CHOIX DES BASES DE DONNÉES

Une base adaptée à chaque type de donnée

MySQL

patient, doctor, appointment

Données relationnelles et transactions : un créneau ne peut être réservé qu'une seule fois.

MongoDB

medical-record, notification

Documents hétérogènes dont le schéma évolue ; journaux volumineux sans relations.

PostgreSQL

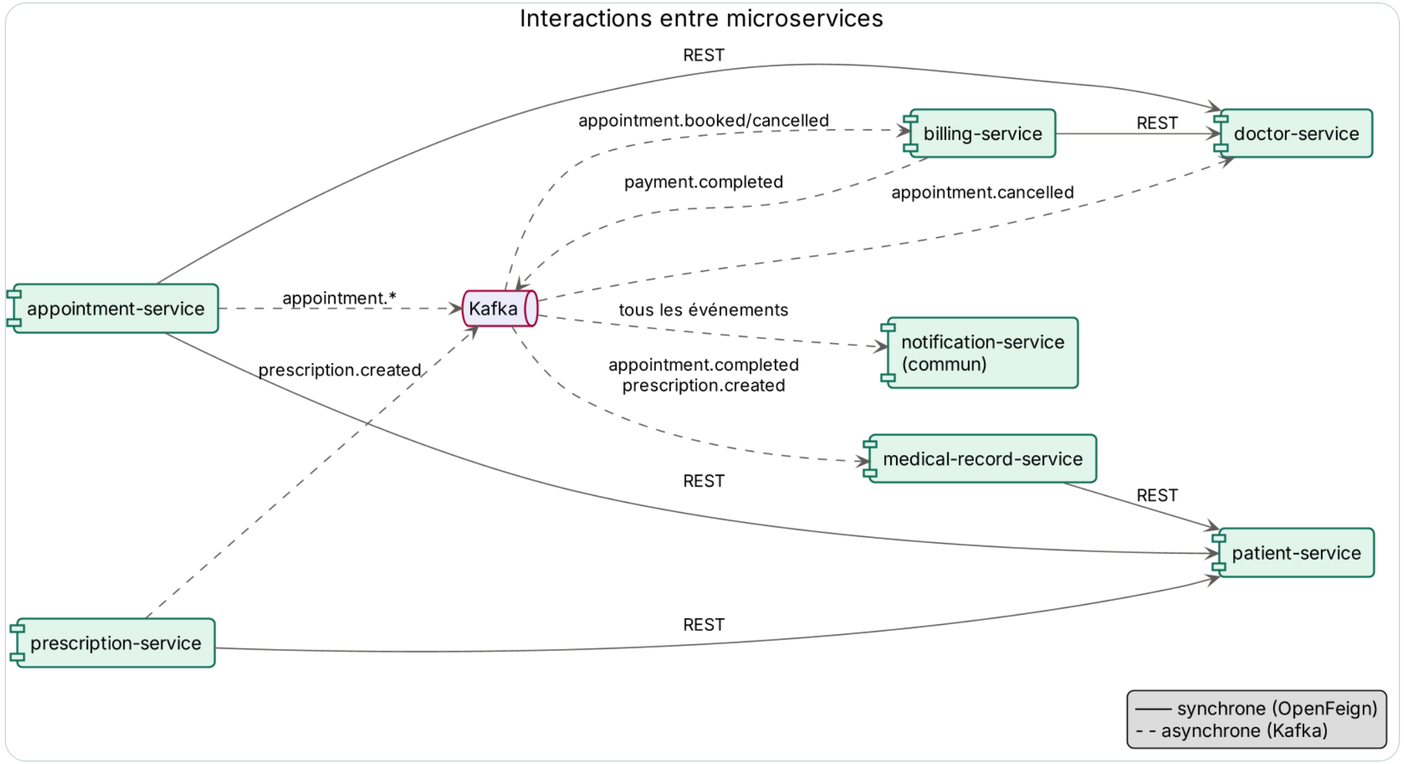
prescription, billing

Données financières et réglementaires : cohérence ACID, montants en NUMERIC, audit.

Database per service : aucun service ne lit la base d'un autre ; les références passent par identifiant (patientId, doctorId).

COMMUNICATIONS ENTRE SERVICES

Synchrone quand il faut une réponse, asynchrone sinon



Synchrone · OpenFeign

appointment → patient, doctor
prescription → patient
billing → doctor (tarif)

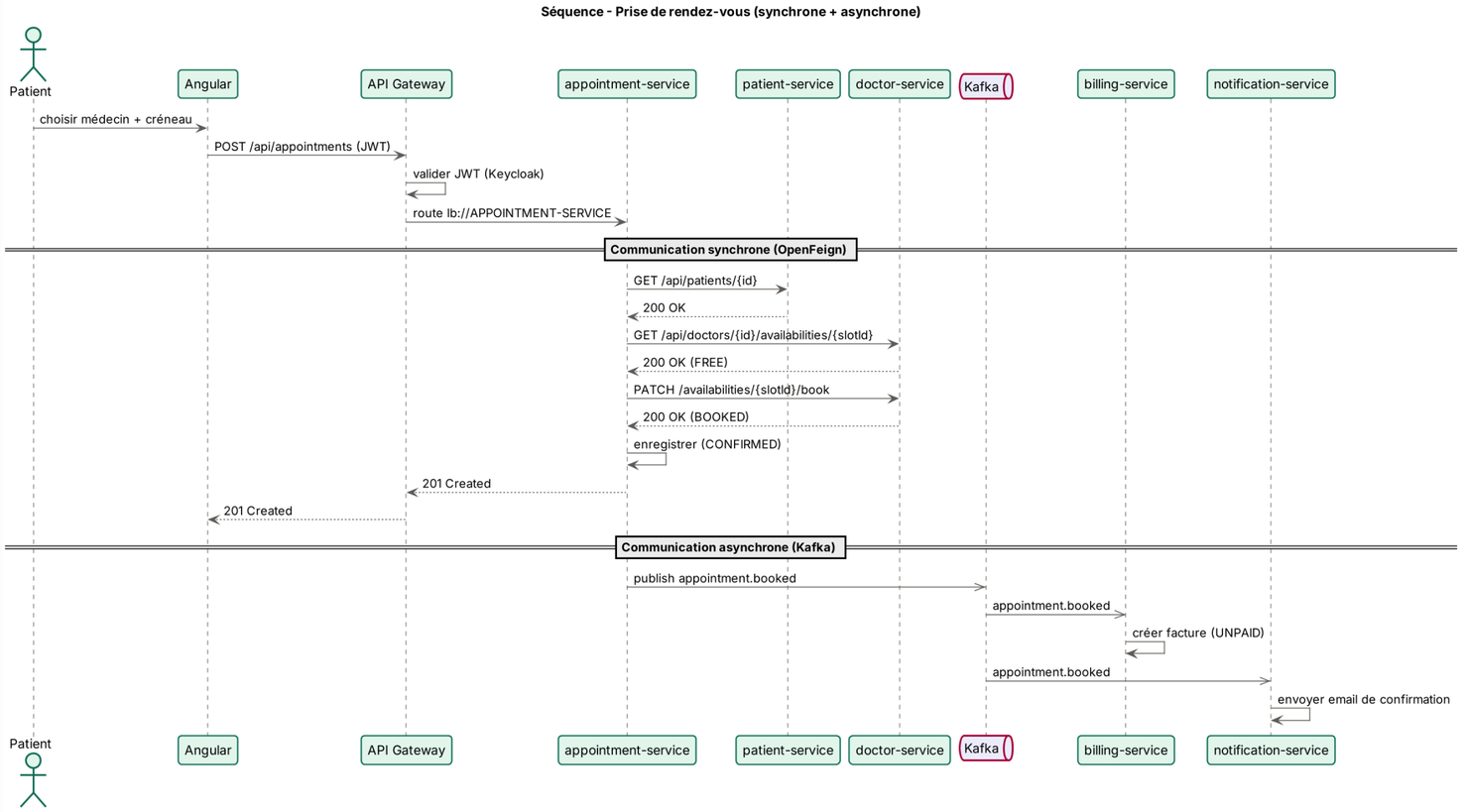
Asynchrone · Kafka

appointment.booked / cancelled / completed
prescription.created
payment.completed

Résilience : Resilience4j (circuit breaker, retry) sur les appels critiques.

SCÉNARIO CLÉ

Prise de rendez-vous de bout en bout

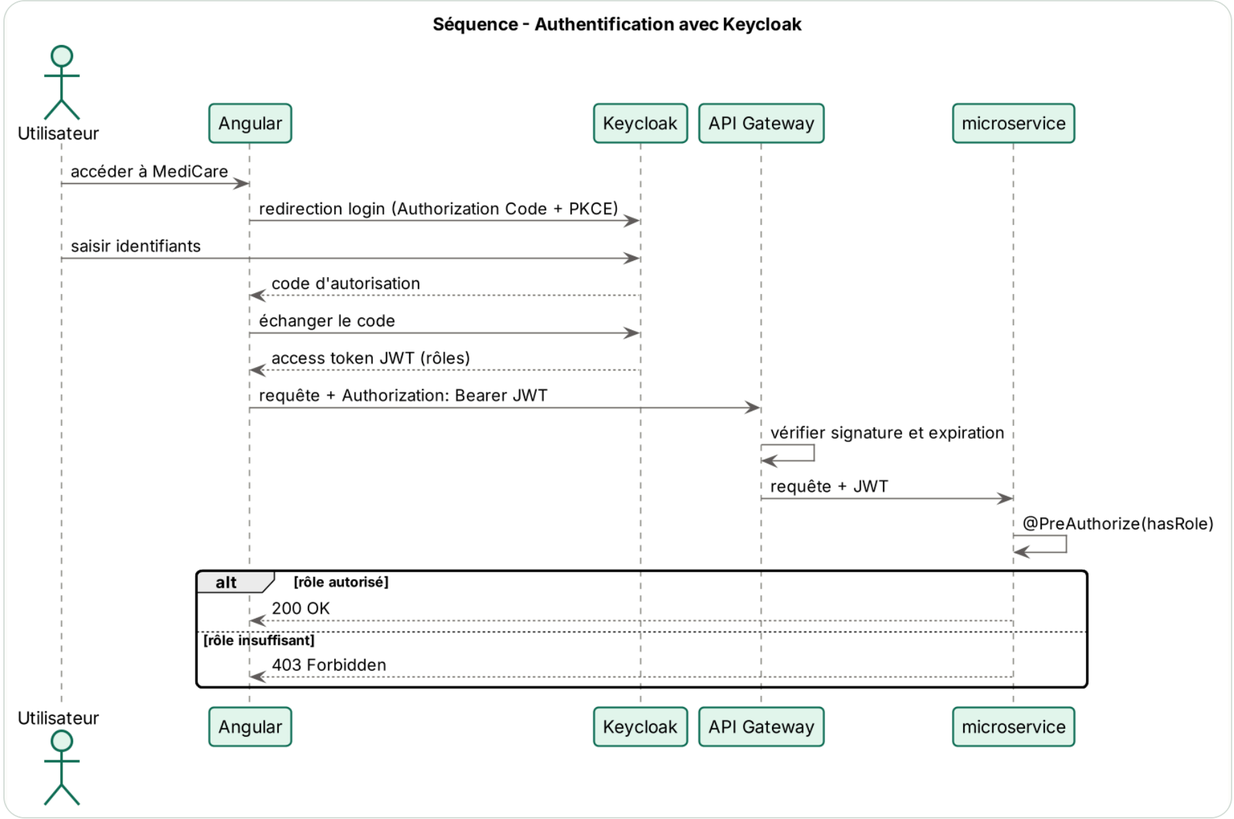


SÉCURITÉ

Keycloak, JWT et rôles

Rôle	Droits principaux
PATIENT	Ses rendez-vous, dossier, factures
DOCTOR	Planning, dossiers, ordonnances
PHARMACIST	Délivrance des ordonnances
ADMIN	Médecins, utilisateurs, facturation

Authorization Code + PKCE depuis Angular, @PreAuthorize sur les méthodes, jeton propagé dans les appels OpenFeign.



TECHNOLOGIES CHOISIES

Une stack vue en cours, cohérente de bout en bout

BACKEND

Java 17, Spring Boot 3

SPRING CLOUD

Gateway, Eureka, Config, OpenFeign, Resilience4j

MESSAGING

Apache Kafka

SÉCURITÉ

Keycloak, OAuth2, JWT

BASES

MySQL, PostgreSQL, MongoDB

FRONTEND

Angular

DEVOPS

Docker, Docker Compose, GitHub

API & GESTION

Swagger, Jira

ORGANISATION DU PROJET

Dépôt Git et gestion des tâches

```
medicare-4dasi14/
├── Backend/ (10 services)
├── Frontend/
├── Documentation/
│   ├── Presentation/
│   ├── Diagrams/
│   └── Dossier-technique/
└── README.md
```

Branches

main stable · **develop** intégration · **feature/<service>-<tâche>** par tâche, fusion par Pull Request relue.

Gestion de projet

Jira en sprints d'une semaine : To Do, In Progress, Review, Done. Chaque ticket est lié à une branche.

RÉPARTITION DES TÂCHES

Un microservice par membre, le reste partagé

Membre	Microservice	Partie commune
Emna Laouini	patient-service	Problématique, acteurs, dépôt Git
Dhouha Belhaj	doctor-service	Fonctionnalités, Eureka + Config
Eslem Zen	appointment-service	Architecture, API Gateway
Sarra Akremi	medical-record-service	Bases de données, Kafka
Hiba Dogui	prescription-service	Technologies, Keycloak
Maram Ghazel	billing-service	Planning, risques, Notification
Toute l’équipe	notification-service	Docker Compose, Frontend, intégration

PLANNING ET RISQUES

Douze semaines, trois risques à surveiller

S1–3	S4	S5–6	S7–8	S9–10	S11–12
Analyse, découpage	Validation conception	Infrastructure, CRUD	OpenFeign, Kafka	Angular, sécurité	Docker, tests, final

Intégration

Contrats d’API définis tôt, Swagger, tests d’intégration.

Kafka et Keycloak

Docker Compose commun et guide interne partagé.

Retards et conflits Git

Suivi Jira hebdomadaire, Pull Requests, merges fréquents.

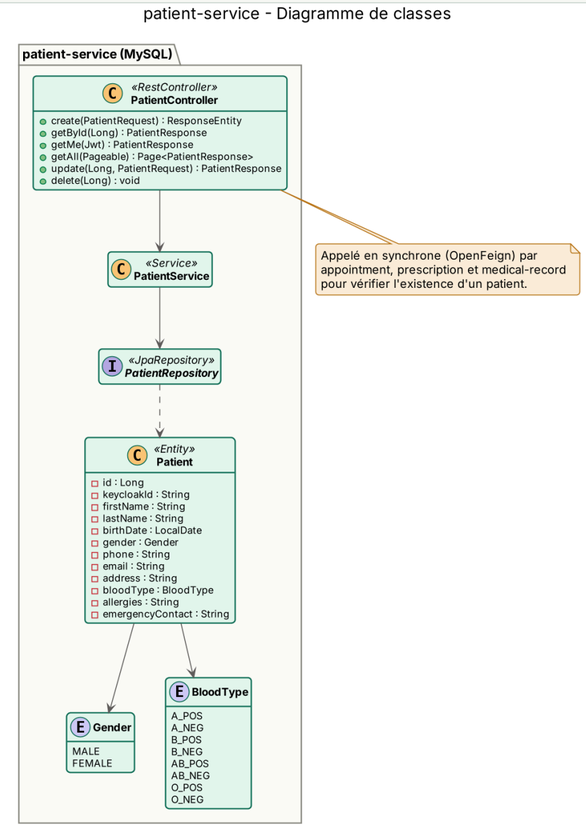
PARTIE INDIVIDUELLE

Les microservices

Conception, interactions et contrats d’API, présentés par chaque responsable en une minute.

EMNA LAOUINI · MYSQL

patient-service



SYNCHRONE

Appelé par appointment, prescription et medical-record pour vérifier un patient.

ASYNCHRONE

Aucun événement : service de référence.

ENDPOINTS PRINCIPAUX

POST `/api/patients` 201 · 400 · 409

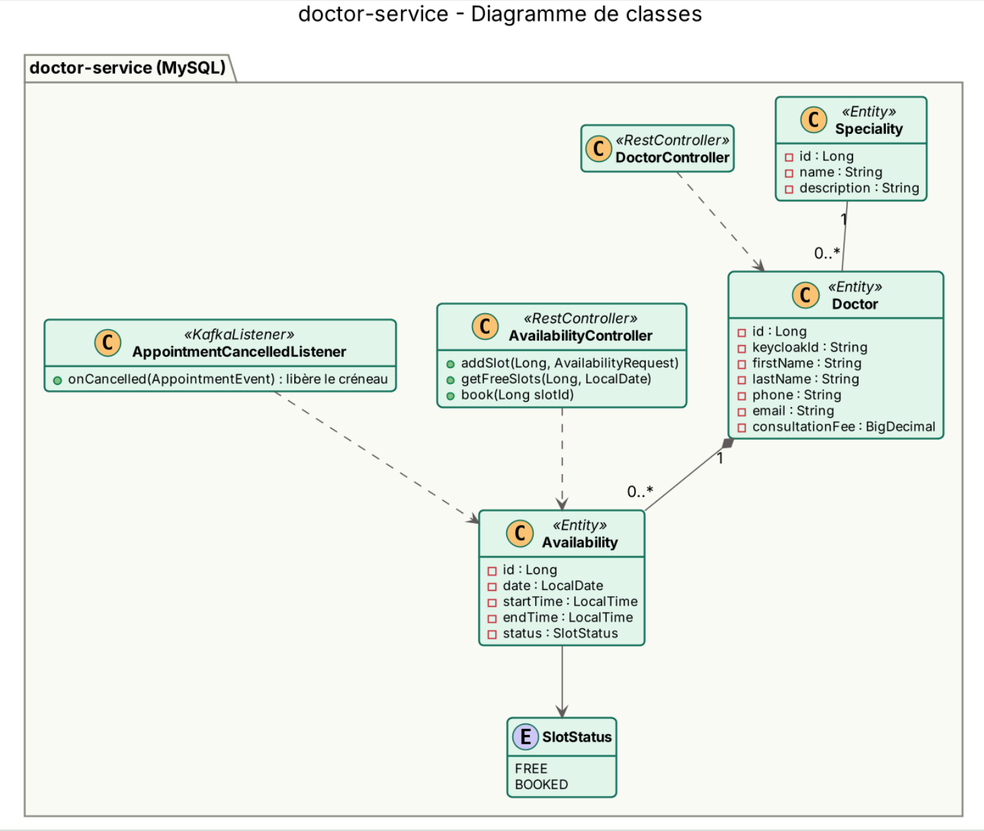
GET `/api/patients/{id}` 200 · 404

GET `/api/patients/me` 200 · 404

PUT `/api/patients/{id}` 200 · 404

DHOUHA BELHAJ · MYSQL

doctor-service



SYNCHRONE

Appelé par appointment (vérifier et réserver un créneau) et par billing (tarif).

ASYNCHRONE

Consomme appointment.cancelled pour libérer le créneau.

ENDPOINTS PRINCIPAUX

POST `/api/doctors` 201 · 400

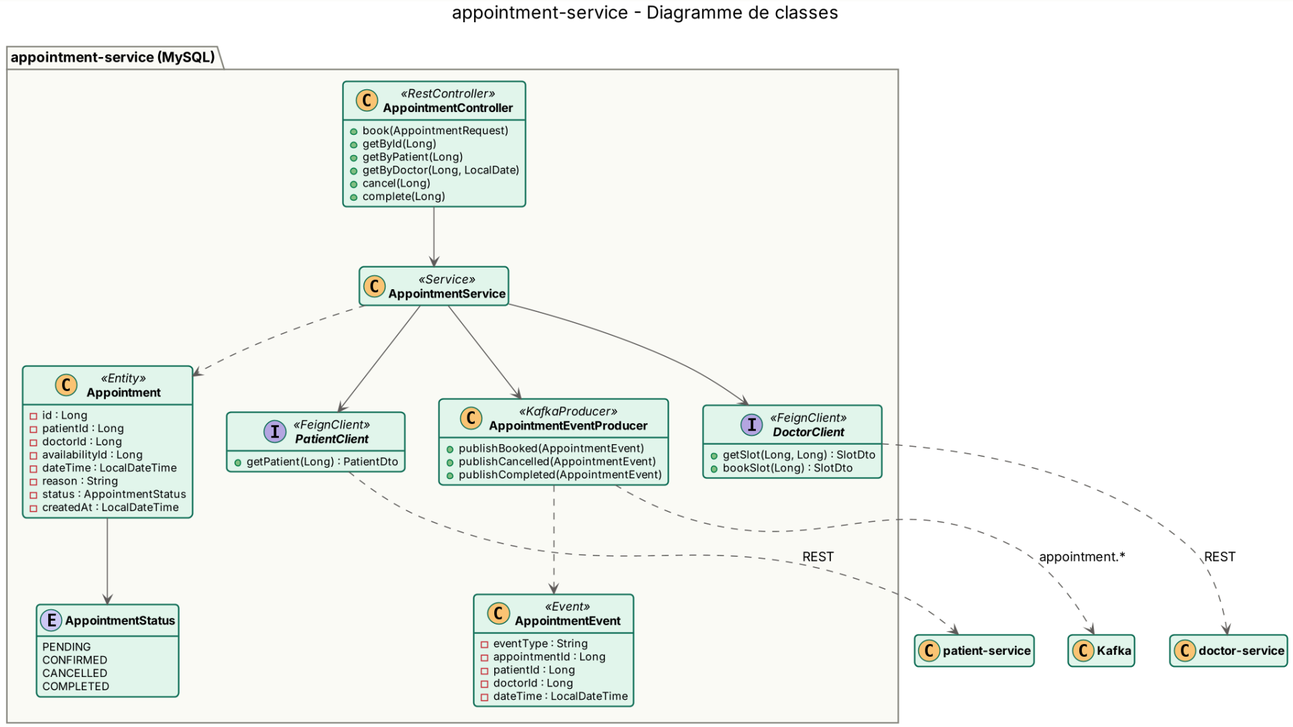
GET `/api/doctors?speciality=` 200

POST `/doctors/{id}/availabilities` 201 · 409

PATCH `/availabilities/{slot}/book` 200 · 409

ESLEM ZEN · MYSQL

appointment-service



SYNCHRONE

Appelle patient-service et doctor-service via OpenFeign (PatientClient, DoctorClient).

ASYNCHRONE

Publie appointment.booked, appointment.cancelled et appointment.completed.

ENDPOINTS PRINCIPAUX

POST `/api/appointments` 201 · 404 · 409

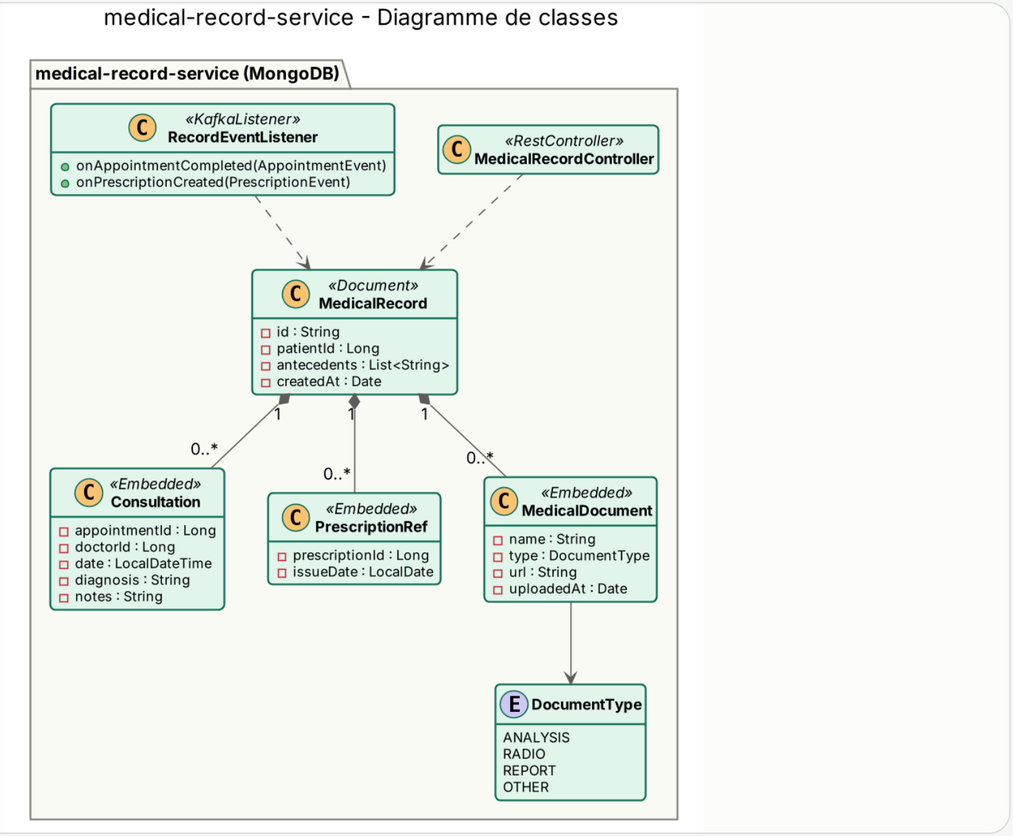
GET `/appointments/patient/{id}` 200

PATCH `/appointments/{id}/cancel` 200 · 409

PATCH `/appointments/{id}/complete` 200

SARRA AKREMI · MONGODB

medical-record-service



SYNCHRONE

Appelle patient-service pour afficher l'identité du patient.

ASYNCHRONE

Consomme appointment.completed et prescription.created.

ENDPOINTS PRINCIPAUX

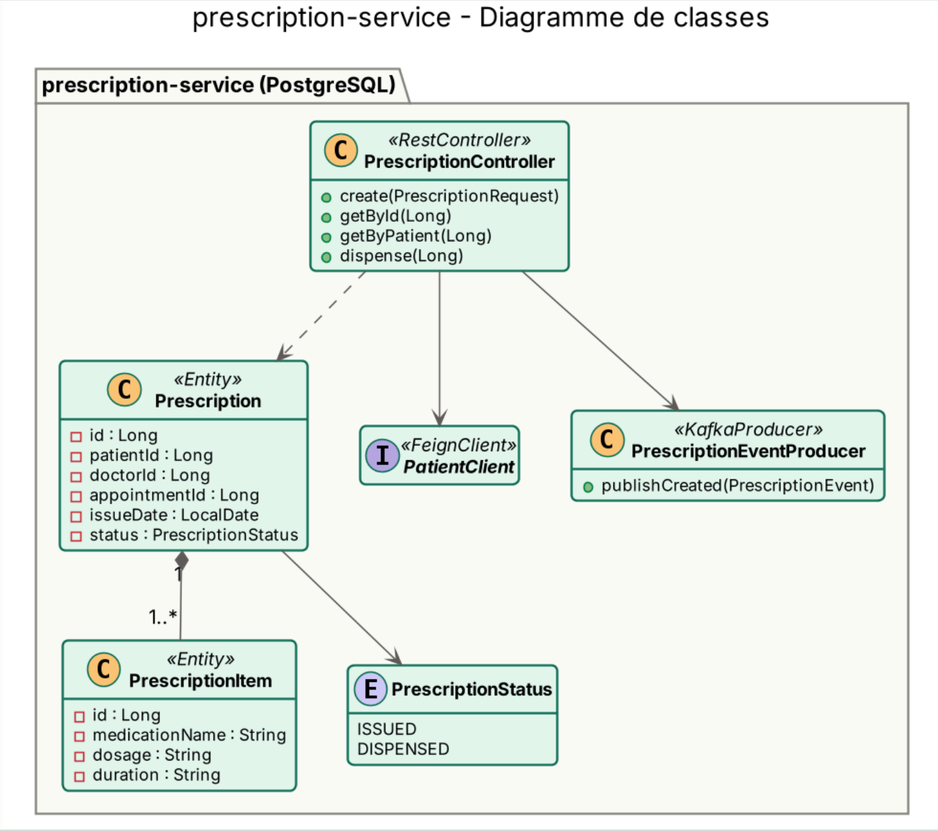
GET `/medical-records/patient/{id}` 200 · 404

POST `/api/medical-records` 201 · 409

POST `/{id}/consultations` 201 · 404

POST `/{id}/documents` 201 · 404

prescription-service



SYNCHRONE

Appelle patient-service pour vérifier le patient avant d'émettre l'ordonnance.

ASYNCHRONE

Publie prescription.created vers medical-record et notification.

ENDPOINTS PRINCIPAUX

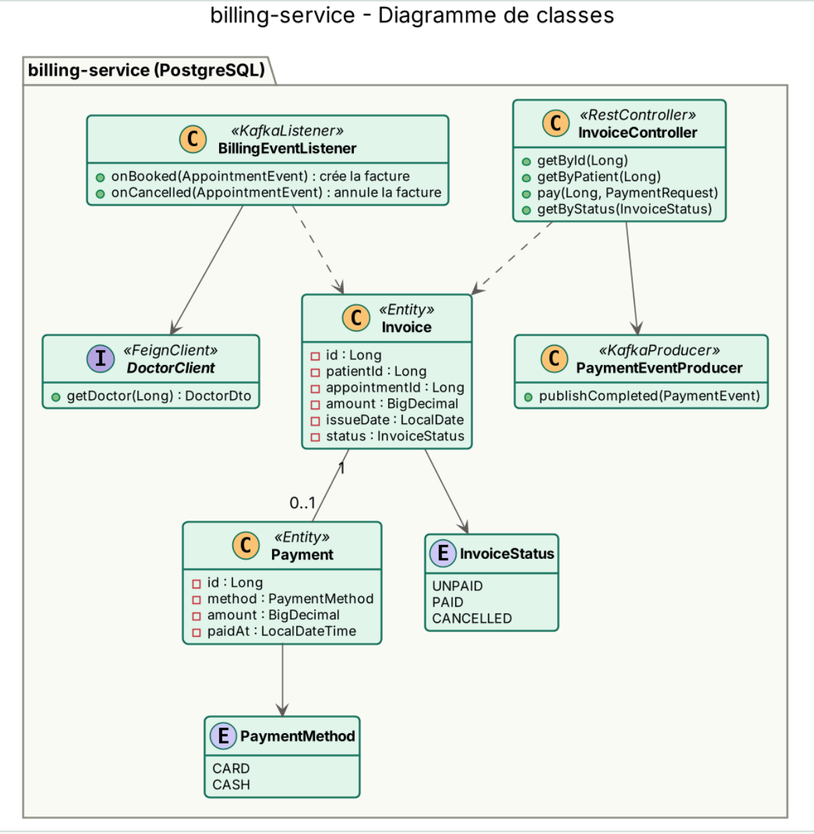
POST `/api/prescriptions` 201 · 400 · 404

GET `/prescriptions/patient/{id}` 200

PATCH `/prescriptions/{id}/dispense` 200 · 409

MARAM GHAZEL · POSTGRESQL

billing-service



SYNCHRONE

Appelle doctor-service pour récupérer le tarif de consultation.

ASYNCHRONE

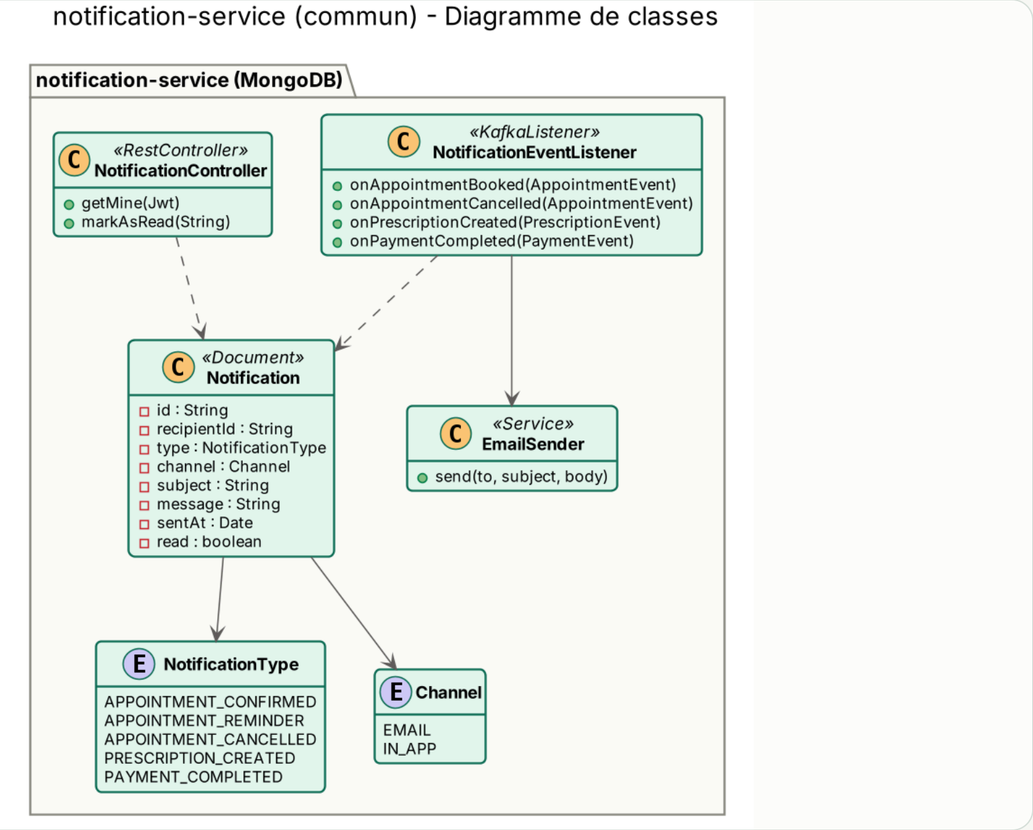
Consomme appointment.booked / cancelled ; publie payment.completed.

ENDPOINTS PRINCIPAUX

```
GET /invoices/patient/{id} 200
GET /api/invoices/{id} 200 · 404
POST /api/invoices/{id}/pay 200 · 409
GET /api/invoices?status= 200
```

MARAM GHAZEL · MONGODB

notification-service (commun)



SYNCHRONE

Aucun appel sortant : service 100 % réactif.

ASYNCHRONE

Consomme appointment.*, prescription.created et payment.completed, puis envoie l'email.

ENDPOINTS PRINCIPAUX

GET `/api/notifications/me` 200

PATCH `/notifications/{id}/read` 200 · 404

Merci

Place à vos questions.

github.com/emnalaouiniii/medicare-4dasi14